

Correlation-Tempered KDE Pooling with Good/Bad Measurement Components

Formalism and implementation plan for J0030 posterior combination

Draft technical note

Purpose. This note defines a practical revised framework for combining published PSR J0030+0451 posterior distributions when the inputs are not independent. The framework keeps the original ingredients of the previous work:

- each measurement is represented by a non-parametric posterior density reconstructed with `scipy.stats.gaussian_kde`;
- each measurement has a good component and a bad component;
- each bad component can shift relative to the good posterior and can be broadened;
- the combined likelihood is correlation-tempered using an effective inter-posterior correlation parameter ρ ;
- a group-level systematic covariance is marginalized by convolution.

The framework is deliberately written first for the one-dimensional compactness case, then extended to the two-dimensional (C, M) case. Here

$$C \equiv \frac{GM}{Rc^2}. \quad (1)$$

The two-dimensional implementation uses $x = (C, M)^T$. Radius can be derived afterward from

$$R = \frac{GM}{C^2}. \quad (2)$$

In solar-mass and km units,

$$R [\text{km}] = \frac{1.476625 (M/M_\odot)}{C}. \quad (3)$$

1 Statistical interpretation

The input objects are not raw likelihoods from independent experiments. They are published posterior samples from different PPM analyses and hotspot models, many of which use overlapping NICER and XMM-Newton data. Therefore, the naive product

$$\prod_{i=1}^N q_i(x) \quad (4)$$

would generally overcount shared information. Here q_i denotes the KDE reconstruction of the i th published posterior.

The revised framework does not claim to reconstruct the unavailable joint NICER likelihood. Instead, it defines a controlled pooling prescription:

$$\log L_\rho(x) = \kappa(\rho) \sum_{i=1}^N w_i \log \ell_i(x), \quad (5)$$

where ℓ_i is the good/bad measurement contribution, w_i is a model weight, and

$$\kappa(\rho) = \frac{1}{1 + (N-1)\rho}, \quad 0 \leq \rho \leq 1. \quad (6)$$

This is an effective-information correction. It has the limits

$$\rho = 0 : \quad \kappa = 1, \quad \text{independent-product endpoint}, \quad (7)$$

$$\rho = 1 : \quad \kappa = 1/N, \quad \text{one-effective-data-set endpoint}. \quad (8)$$

Thus the old product result is not adopted as the main result; it is the $\rho = 0$ limiting case.

The group-level systematic covariance is then incorporated by convolving the pooled likelihood with a systematic kernel. The final posterior is

$$p(x | D) \propto \pi(x) \int_0^1 d\rho \int d\Sigma_{\text{sys}} \pi(\rho) \pi(\Sigma_{\text{sys}}) \int dx' \mathcal{N}(x - x'; 0, \Sigma_{\text{sys}}) \left[\prod_{i=1}^N \ell_i(x')^{w_i} \right]^{\kappa(\rho)}. \quad (9)$$

Equation (9) is the central formula. In one dimension, $x = C$ and $\Sigma_{\text{sys}} = \sigma_{C,\text{sys}}^2$. In two dimensions, $x = (C, M)^T$ and Σ_{sys} is a 2×2 covariance matrix.

2 One-dimensional compactness formalism

2.1 Input posterior KDE

For measurement i , suppose the published posterior samples give compactness samples

$$\{C_{ij}\}_{j=1}^{n_i}. \quad (10)$$

The good component is the KDE density

$$q_i(C) = \text{KDE}[\{C_{ij}\}](C). \quad (11)$$

Operationally, in Python,

```
from scipy.stats import gaussian_kde
kde_i = gaussian_kde(C_samples_i)      # C_samples_i shape: (n_i,)
q_i_values = kde_i(C_grid)             # C_grid shape: (n_grid,)
logq_i = np.log(q_i_values + floor)
```

A small numerical floor is useful to avoid $\log(0)$ in grid regions where the KDE evaluates to extremely small values.

2.2 Measurement bad component with shift and broadening

The bad component is a broadened, shifted Gaussian surrogate:

$$B_i(C | \alpha_i, \Delta_i) = \mathcal{N}(C; \mu_i + \Delta_i, \alpha_i^2 S_i^2), \quad (12)$$

where

- μ_i is the median of the compactness samples;
- S_i is a baseline bad-distribution width;
- $\alpha_i \geq 1$ is a broadening parameter;
- Δ_i is a measurement-specific systematic shift.

A robust choice for S_i is based on a central credible interval of the good posterior. For example, let $w_i(q_G)$ be the half-width of the central q_G interval around μ_i . Then choose

$$S_i = \frac{w_i(q_G)}{z_{q_B}}, \quad z_{q_B} = \Phi^{-1} \left(\frac{1 + q_B}{2} \right), \quad (13)$$

so that the central q_B interval of the bad Gaussian covers roughly the central q_G interval of the good posterior. A useful setting is $q_G = 0.99$ and $q_B = 0.68$.

Possible priors are

$$\alpha_i - 1 \sim \text{Exponential}(\lambda_\alpha), \quad (14)$$

$$\Delta_i \sim \mathcal{N}(0, \sigma_{\Delta,i}^2), \quad (15)$$

$$\sigma_{\Delta,i} \sim \text{HalfNormal}(s_{\Delta,i}). \quad (16)$$

The exact prior scale $s_{\Delta,i}$ should be broad enough to permit shifts comparable to the observed model-to-model compactness scatter.

2.3 Good/bad measurement likelihood contribution

The measurement contribution is a mixture:

$$\ell_i(C \mid p_i, \alpha_i, \Delta_i) = p_i q_i(C) + (1 - p_i) B_i(C \mid \alpha_i, \Delta_i), \quad (17)$$

where p_i is the probability that measurement i is represented by its good KDE component. One may use a common p for all measurements or independent p_i values. A simple prior is

$$p_i \sim \text{Beta}(a_p, b_p). \quad (18)$$

For a neutral setting use $\text{Beta}(1, 1)$. For a prior that mildly favors the published KDE unless the ensemble requires a bad component, use $\text{Beta}(2, 1)$ or $\text{Beta}(4, 1)$.

For numerical stability, evaluate Eq. (17) in log space:

$$\log \ell_i(C) = \text{logsumexp} [\log p_i + \log q_i(C), \log(1 - p_i) + \log B_i(C)]. \quad (19)$$

2.4 Correlation-tempered compactness pooling

For N compactness posteriors, define

$$\log L_\rho(C) = \kappa(\rho) \sum_{i=1}^N w_i \log \ell_i(C), \quad (20)$$

with $\kappa(\rho)$ given by Eq. (6). If all measurements have equal weight, set $w_i = 1$. If evidence weights are used, normalize them so that

$$\sum_{i=1}^N w_i = N. \quad (21)$$

This normalization ensures that the effective total information scale is controlled by $\kappa(\rho)$ rather than by the arbitrary normalization of the weights.

Recommended ρ priors:

$$\rho \sim \text{Beta}(4, 1) \quad \text{main conservative high-correlation prior}, \quad (22)$$

$$\rho \sim \text{Uniform}(0, 1) \quad \text{sensitivity prior}, \quad (23)$$

$$\rho \sim \text{Beta}(1, 4) \quad \text{low-correlation stress test}, \quad (24)$$

$$\rho = 0 \quad \text{old independent-product endpoint}, \quad (25)$$

$$\rho = 1 \quad \text{fully correlated one-effective-data-set endpoint}. \quad (26)$$

2.5 Group-level systematic broadening in 1D

The compactness-level group systematic is represented by

$$C' = C + b_C, \quad b_C \sim \mathcal{N}(0, \sigma_{C,\text{sys}}^2). \quad (27)$$

Equivalently, the pooled likelihood is convolved with a Gaussian systematic kernel:

$$\tilde{L}(C \mid \rho, \sigma_{C,\text{sys}}) = \int dC' \mathcal{N}(C - C'; 0, \sigma_{C,\text{sys}}^2) L_\rho(C'). \quad (28)$$

Use a half-normal prior

$$\sigma_{C,\text{sys}} \sim \text{HalfNormal}(s_C), \quad (29)$$

where a data-anchored scale is

$$s_C = \text{Std}[\text{median}(C_i)], \quad (30)$$

with the standard deviation taken over the input posteriors' medians.

The final 1D posterior is

$$p(C \mid D) \propto \pi(C) \int d\rho d\sigma_{C,\text{sys}} d\eta \pi(\rho) \pi(\sigma_{C,\text{sys}}) \pi(\eta) \tilde{L}(C \mid \rho, \sigma_{C,\text{sys}}, \eta), \quad (31)$$

where $\eta = \{p_i, \alpha_i, \Delta_i\}_{i=1}^N$.

3 One-dimensional pseudocode

Inputs:

```
C_samples_list = [C_samples_1, ..., C_samples_N]
C_grid         = 1D grid covering all input supports
N_hyper        = number of hyperparameter draws
weights        = model weights, normalized so sum(weights)=N
```

Precompute per-measurement objects:

```
for i in range(N):
    kde_i = gaussian_kde(C_samples_i)
    q_i_grid = kde_i(C_grid)
    logq_i_grid = log(q_i_grid + floor)

    mu_i = median(C_samples_i)
    S_i = baseline_bad_width(C_samples_i)
    store mu_i, S_i, logq_i_grid
```

Initialize:

```
posterior_grid = zeros_like(C_grid)
```

Monte Carlo marginalization:

```
for s in range(N_hyper):
    rho = sample_rho_prior()          # e.g. Beta(4,1)
    kappa = 1.0 / (1.0 + (N-1)*rho)

    sigma_C_sys = sample_halfnormal(s_C)

    logL_sum = zeros_like(C_grid)

    for i in range(N):
```

```

p_i      = sample_beta(a_p, b_p)
alpha_i  = 1.0 + sample_exponential(lambda_alpha)
Delta_i  = sample_normal(0.0, sigma_Delta_i)

logB_i = normal_logpdf(
    C_grid,
    mean = mu_i + Delta_i,
    sd    = alpha_i * S_i
)

logell_i = logaddexp(
    log(p_i)      + logq_i_grid[i],
    log(1-p_i)    + logB_i
)

logL_sum += weights[i] * logell_i

logL = kappa * logL_sum
L = exp(logL - max(logL))

L_conv = gaussian_convolve_1d(
    L,
    grid_spacing = C_grid[1]-C_grid[0],
    sigma = sigma_C_sys
)

posterior_grid += L_conv

```

Finalize:

```

posterior_grid /= N_hyper
posterior_grid *= prior_C_grid
posterior_grid /= trapz(posterior_grid, C_grid)

```

Output:

```

posterior density p(C|D)
posterior samples drawn from C_grid using posterior_grid

```

4 Two-dimensional (C, M) formalism

4.1 Input KDEs in (C, M)

For posterior i , construct samples

$$x_{ij} = \begin{pmatrix} C_{ij} \\ M_{ij} \end{pmatrix}. \quad (32)$$

The good component is

$$q_i(x) = \text{KDE} \left[\{x_{ij}\}_{j=1}^{n_i} \right] (x). \quad (33)$$

Operationally, `gaussian_kde` expects shape (d, n) :

```

X_i = np.vstack([C_samples_i, M_samples_i]) # shape: (2, n_i)
kde_i = gaussian_kde(X_i)

```

```

CM_points = np.vstack([C_grid_flat, M_grid_flat]) # shape: (2, n_grid)
q_i_flat = kde_i(CM_points)

```

```
logq_i_flat = np.log(q_i_flat + floor)
logq_i_grid = logq_i_flat.reshape(nC, nM)
```

The grid should cover the union of the supports of all input posteriors, with margin large enough for systematic convolution.

4.2 Two-dimensional measurement bad component

The two-dimensional bad component is

$$B_i(x \mid \alpha_i, \Delta_i) = \mathcal{N}_2(x; \mu_i + \Delta_i, \alpha_i^2 \Sigma_{B,i}), \quad (34)$$

where

$$\mu_i = \begin{pmatrix} \text{median}(C_i) \\ \text{median}(M_i) \end{pmatrix}, \quad \Delta_i = \begin{pmatrix} \Delta_{C,i} \\ \Delta_{M,i} \end{pmatrix}. \quad (35)$$

Use a full covariance for $\Sigma_{B,i}$ rather than a diagonal covariance. A practical choice is

$$\Sigma_{B,i} = s_{B,i}^2 \widehat{\text{Cov}}_i(C, M), \quad (36)$$

where $\widehat{\text{Cov}}_i(C, M)$ is the sample covariance or a robust covariance estimated from the central 90–99% of posterior samples. The scalar $s_{B,i} \geq 1$ can be chosen so that the bad distribution is broader than the good posterior. One may also absorb this factor into α_i and set $s_{B,i} = 1$.

The shift prior can be

$$\Delta_i \sim \mathcal{N}_2(0, \Sigma_{\Delta,i}), \quad (37)$$

with $\Sigma_{\Delta,i}$ broad enough to allow shifts comparable to inter-model scatter.

4.3 Two-dimensional good/bad measurement contribution

The measurement contribution is

$$\ell_i(x \mid p_i, \alpha_i, \Delta_i) = p_i q_i(x) + (1 - p_i) B_i(x \mid \alpha_i, \Delta_i). \quad (38)$$

The pooled likelihood is

$$\log L_\rho(x) = \kappa(\rho) \sum_{i=1}^N w_i \log \ell_i(x). \quad (39)$$

4.4 Two-dimensional group-level systematic covariance

The group-level systematic covariance is

$$\Sigma_{\text{sys}} = D_{\text{sys}} R_{\text{sys}} D_{\text{sys}}, \quad (40)$$

with

$$D_{\text{sys}} = \begin{pmatrix} \sigma_{C,\text{sys}} & 0 \\ 0 & \sigma_{M,\text{sys}} \end{pmatrix}, \quad (41)$$

and

$$R_{\text{sys}} = \begin{pmatrix} 1 & r_{\text{sys}} \\ r_{\text{sys}} & 1 \end{pmatrix}. \quad (42)$$

Use priors

$$\sigma_{C,\text{sys}} \sim \text{HalfNormal}(s_C), \quad (43)$$

$$\sigma_{M,\text{sys}} \sim \text{HalfNormal}(s_M), \quad (44)$$

$$r_{\text{sys}} \sim \text{Uniform}(-1, 1), \quad (45)$$

where

$$s_C = \text{Std}[\text{median}(C_i)], \quad (46)$$

$$s_M = \text{Std}[\text{median}(M_i)]. \quad (47)$$

A more formal version can use an LKJ prior for R_{sys} , but for a two-dimensional implementation the scalar correlation prior above is sufficient and transparent.

The systematic-convolved likelihood is

$$\tilde{L}(x \mid \rho, \Sigma_{\text{sys}}) = \int d^2x' \mathcal{N}_2(x - x'; 0, \Sigma_{\text{sys}}) L_\rho(x'). \quad (48)$$

The final posterior is Eq. (9) with $x = (C, M)^T$.

5 Two-dimensional pseudocode

Inputs:

```
samples_list = [(C_1, M_1), ..., (C_N, M_N)]
C_grid, M_grid
N_hyper
weights normalized so sum(weights)=N
```

Create grid:

```
CC, MM = np.meshgrid(C_grid, M_grid, indexing='ij')
points = np.vstack([CC.ravel(), MM.ravel()])
```

Precompute KDEs and bad-component baselines:

```
for i in range(N):
    X_i = np.vstack([C_samples_i, M_samples_i])
    kde_i = gaussian_kde(X_i)

    q_i_flat = kde_i(points)
    logq_i_grid[i] = log(q_i_flat + floor).reshape(nC, nM)

    mu_i = [median(C_samples_i), median(M_samples_i)]
    Sigma_B_i = robust_covariance(X_i)
    Sigma_Delta_i = broad_shift_covariance_from_scatter(...)
```

Initialize:

```
posterior_grid = zeros((nC, nM))
```

Monte Carlo marginalization:

```
for s in range(N_hyper):
    rho = sample_rho_prior() # main: Beta(4,1)
    kappa = 1.0 / (1.0 + (N-1)*rho)

    sigma_C_sys = sample_halfnormal(s_C)
    sigma_M_sys = sample_halfnormal(s_M)
    r_sys = sample_uniform(-1.0, 1.0)
    Sigma_sys = [[sigma_C_sys**2,
                  r_sys*sigma_C_sys*sigma_M_sys],
                [r_sys*sigma_C_sys*sigma_M_sys,
                 sigma_M_sys**2]]

    logL_sum = zeros((nC, nM))
```

```

for i in range(N):
    p_i      = sample_beta(a_p, b_p)
    alpha_i = 1.0 + sample_exponential(lambda_alpha)
    Delta_i = sample_multivariate_normal(0, Sigma_Delta_i)

    Sigma_bad = alpha_i**2 * Sigma_B_i
    logB_i = mvn_logpdf_grid(
        CC, MM,
        mean = mu_i + Delta_i,
        cov   = Sigma_bad
    )

    logell_i = logaddexp(
        log(p_i) + logq_i_grid[i],
        log(1-p_i) + logB_i
    )

    logL_sum += weights[i] * logell_i

logL = kappa * logL_sum
L = exp(logL - max(logL))

L_conv = convolve_2d_with_gaussian_covariance(
    L,
    C_grid,
    M_grid,
    Sigma_sys
)

posterior_grid += L_conv

Finalize:
    posterior_grid /= N_hyper
    posterior_grid *= prior_CM_grid
    posterior_grid /= sum(posterior_grid) * dC * dM

```

Output:

```

p(C,M|D) on grid
posterior samples in (C,M)
transformed samples in (M,R), with R = 1.476625*M/C km

```

6 Operational details for Gaussian KDEs

6.1 KDE evaluation

For one dimension:

```

kde = gaussian_kde(C_samples)
logq = np.log(kde(C_grid) + floor)

```

For two dimensions:

```

X = np.vstack([C_samples, M_samples])
kde = gaussian_kde(X)
points = np.vstack([CC.ravel(), MM.ravel()])

```

```
logq = np.log(kde(points) + floor).reshape(nC, nM)
```

6.2 Bandwidth

The default `gaussian_kde` bandwidth is Scott’s rule. To match the existing paper, use the same bandwidth prescription consistently across all inputs. If the existing analysis uses the Silverman factor, set it explicitly:

```
kde = gaussian_kde(X, bw_method='silverman')
```

For reproducibility, record the exact bandwidth rule in the paper and in the public code.

6.3 Prior correction

Strictly, a posterior is

$$q_i(x) \propto L_i(x)\pi_i(x). \quad (49)$$

If the input analyses used common or approximately flat priors over the posterior support, $q_i(x)$ can be used as a likelihood surrogate up to normalization. If not, one should correct by

$$L_i(x) \propto \frac{q_i(x)}{\pi_i(x)}. \quad (50)$$

This correction should be discussed explicitly. If the published priors cannot be reconstructed, the analysis should be described as posterior pooling rather than exact likelihood combination.

7 Recommended diagnostic outputs

The revised analysis should show at least the following curves or contours:

1. **Old product endpoint:** $\rho = 0$, $\Sigma_{\text{sys}} = 0$. This is the previous overconfident limit.
2. **Fully correlated endpoint:** $\rho = 1$, $\Sigma_{\text{sys}} = 0$.
3. **Correlation-marginalized result:** $\rho \sim \text{Uniform}(0, 1)$, no group systematic.
4. **Conservative main result:** $\rho \sim \text{Beta}(4, 1)$ with Σ_{sys} marginalized.
5. **Model-weight sensitivity:** equal weights versus within-analysis evidence weights where evidence comparisons are meaningful.

8 Suggested language for the manuscript

A safe description is:

We introduce a correlation-tempered hierarchical pooling model for published J0030+0451 posterior samples. Each input posterior is represented non-parametrically by a Gaussian KDE and enters through a good/bad mixture component. To avoid interpreting overlapping NICER/XMM analyses as independent measurements, the pooled log-likelihood is tempered by an effective inter-posterior correlation parameter ρ , which is marginalized over. The original product of posteriors is recovered only as the limiting case $\rho = 0$ and is shown as a diagnostic endpoint rather than as the adopted source-level posterior. We additionally marginalize over a group-level systematic covariance, implemented as a convolution of the pooled likelihood, to account for residual PPM model systematics.

Another important caveat is:

This construction is not a reconstruction of the unavailable joint likelihood of the shared NICER/XMM data. It is a controlled sensitivity model for propagating unknown inter-posterior correlation and model-dependent systematic uncertainty into a source-level summary posterior.

9 Minimal implementation checklist

1. Convert all input posterior samples to compactness C and mass M .
2. Build 1D KDEs $q_i(C)$ and test the compactness-only implementation first.
3. Add good/bad mixture components $\ell_i(C)$ with Δ_i and α_i .
4. Add correlation tempering with $\kappa(\rho) = 1/[1 + (N - 1)\rho]$.
5. Add 1D group systematic convolution with $\sigma_{C,\text{sys}}$.
6. Validate normalization and recover the old result when $\rho = 0$ and $\sigma_{C,\text{sys}} = 0$.
7. Extend to 2D (C, M) KDEs $q_i(C, M)$.
8. Use full covariance bad distributions $B_i(C, M)$.
9. Add 2D group systematic convolution with Σ_{sys} .
10. Transform final (C, M) samples to (M, R) for presentation.
11. Report sensitivity to ρ priors and model weights.

10 Compact pseudocode summary

```

for each posterior i:
    build q_i with scipy.stats.gaussian_kde
    compute median mu_i
    compute baseline bad covariance S_i or Sigma_B_i

for hyper draw s:
    draw rho
    draw systematic covariance Sigma_sys
    for each measurement i:
        draw p_i
        draw alpha_i
        draw shift Delta_i
        evaluate good density log q_i on grid
        evaluate bad density log B_i on grid
        log ell_i = logaddexp(log p_i + log q_i,
                             log(1-p_i) + log B_i)
    log pooled = kappa(rho) * sum_i w_i * log ell_i
    pooled = exp(log pooled - max(log pooled))
    pooled_sys = Gaussian_convolution(pooled, Sigma_sys)
    add pooled_sys to posterior accumulator

normalize accumulator
sample final posterior

```